

PROJECT DOCUMENTATION — Result Management System (Ruhuna EngRMS)

Location: `G:\ResultManagementSystem` Written: 2026-10-03, based on a full read of the source code, config files, and docs folder. Rule used: every fact below has a file path. If something was not in the code, it says **"Not found in the project"**.

1. PROJECT OVERVIEW

1.1 What is this project?

This is a **Result Management System** for the Faculty of Engineering, University of Ruhuna (brand name "Ruhuna EngRMS", seen in `frontend/src/layouts/AdminLayout.jsx` and `student-mobile/app.json`).

The problem it solves (from `docs/system_documentation.md` and `docs/Progress.md`): - Universities must register hundreds of students per batch, manage modules, collect exam results, calculate GPA/SGPA/CGPA, and publish results to students. - This system does all of that: batch creation with auto-generated student passwords, CSV-based result upload, automatic GPA calculation, result publishing with notifications, exam timetables, and examiner duty assignment.

1.2 Who are the users?

User	App they use	Main things they can do
Admin	<code>frontend/</code> (web)	Create/edit/delete batches, allocate students to departments via CSV, manage modules, upload & publish results, view master result sheet, manage examiners and exam schedules
Examiner	<code>frontend/</code> (web)	Register own account, log in, view assigned exam schedule, view notifications, edit profile/password
Student	<code>student-frontend/</code> (web) and <code>student-mobile/</code> (Expo app)	Log in with registration number, forced password change on first login, view results per semester, SGPA/CGPA, exam timetable, notifications

Source: routes in `backend/src/routes/*.js`, pages in `frontend/src/pages/`, `student-frontend/src/pages/`, screens in `student-mobile/src/screens/`.

1.3 Folder structure

```
G:\ResultManagementSystem
├─ backend/                                # Fastify (Node.js) REST API + Prisma ORM
│  └─ src/server.js                        # Entry point, plugin setup, JWT auth hook, department seeding
│  └─ src/lib/prisma.js                    # Prisma client wired to Neon serverless Postgres
│  └─ src/routes/authRoutes.js            # Login / register / change-password endpoints
│  └─ src/routes/adminRoutes.js           # All admin endpoints (batches, modules, results, examiners)
│  └─ src/routes/studentRoutes.js         # Student dashboard, notifications, exam schedules
│  └─ src/routes/examinerRoutes.js        # Examiner schedules, notifications, profile
│  └─ prisma/schema.prisma                # Database schema (10 models)
│  └─ prisma/dev.db                       # Leftover SQLite file (schema now targets Postgres – see §6)
│  └─ test/auth.test.js                   # Vitest + Supertest tests for auth endpoints
│  └─ regenerate_passwords.cjs            # One-off script: regenerate student passwords to CSV
│  └─ Dockerfile                          # node:20-alpine production image
│  └─ .env                                # Contains DATABASE_URL (value not printed here)
├─ frontend/                              # Admin + Examiner web portal (React 19 + Vite)
│  └─ src/pages/Login.jsx                  # Shared login page (Admin / Examiner toggle)
│  └─ src/pages/admin/AdminDashboard.jsx   # Admin orchestrator page
│  └─ src/pages/examiner/ExaminerDashboard.jsx # Examiner portal page
│  └─ src/layouts/AdminLayout.jsx          # Admin sidebar shell
│  └─ src/components/                     # Feature components (batches, modules, results, examiners)
│  └─ src/components/ui/                   # 12 shadcn-style primitives (Base UI based)
│  └─ Dockerfile, nginx.conf               # nginx static hosting (no /api proxy – see §11)
│  └─ vite.config.js, package.json
├─ student-frontend/                      # Student web portal (React 19 + Vite)
│  └─ src/pages/Login.jsx
│  └─ src/pages/student/StudentDashboard.jsx
│  └─ src/pages/student/ChangePassword.jsx
│  └─ src/components/ui/                   # shadcn-style primitives
│  └─ Dockerfile, nginx.conf, vite.config.js
├─ student-mobile/                       # Student mobile app (Expo / React Native)
│  └─ App.js                              # Font loading + native-stack navigator
│  └─ app.json                            # Expo config ("Ruhuna EngRMS")
│  └─ src/screens/                        # LoginScreen, ChangePasswordScreen, DashboardScreen
│  └─ src/components/                     # BrandButton, BrandInput, Card, Badge, SidebarDrawer
│  └─ src/api.js, config.js, theme.js, toast.js, utils.js
│  └─ assets/fonts/Geist-Variable.ttf
│  └─ AGENTS.md                           # AI-agent instructions (mentions Expo Router – not used)
├─ docs/
│  └─ Progress.md                         # Milestone log (Phases 1–5) + backlog
│  └─ system_documentation.md             # Architecture overview doc
│  └─ qa_documentation.md                 # QA/testing strategy doc
│  └─ ResultManagementSystem_Documentation.docx
├─ docker-compose.yml                     # Runs backend + both web apps
├─ 25th_Batch_Passwords (1).csv           # Generated student credentials (PLAINTEXT – see §11)
└─ 26th_Batch_Passwords.csv              # Generated student credentials (PLAINTEXT – see §11)
```

2. TECH STACK

Web frontend — admin portal (`frontend/package.json`) and student portal (`student-frontend/package.json`)

Technology	Version	Why it is used
React	^19.2.8	UI library for component-based pages
react-dom	^19.2.8	Renders React in the browser
react-router-dom	^7.18.4	Client-side routing (<code>BrowserRouter</code>)
Vite	^8.3.0	Dev server and production bundler
@vitejs/plugin-react	^6.1.1	React fast-refresh support in Vite

Technology	Version	Why it is used
Tailwind CSS	^4.3.3 (+ @tailwindcss/vite)	Utility-first styling; brand theme in <code>src/index.css</code>
tw-animate-css	^1.4.0	Animation utility classes
shadcn	^4.21.0	Component generation toolchain (<code>components.json</code> , style "base-nova")
@base-ui/react	^1.8.0	Accessible primitives used by the generated <code>ui/</code> components
class-variance-authority	^0.7.1	Variant styling (<code>cva</code>) inside UI primitives
cn	^0.4.0	Classname merge helper (<code>src/lib/utils.js</code>)
axios	^1.20.0	HTTP client for all backend calls
react-hot-toast	^2.6.1	Toast notifications
lucide-react	^1.47.0	Icons
@fontsource-variable/geist	^5.3.0	Geist variable font
eslint	^1.81.0 (dev)	Linter (<code>npm run lint</code>)
supertest/vitest	—	Not used in frontends (backend only)

Charts library: **Not found in the project** (no recharts etc., although `frontend/src/index.css` defines `--color-chart-*` variables).

Mobile app (`student-mobile/package.json`)

Technology	Version	Why it is used
Expo	~57.0.26	Managed React Native toolchain (no android/ios folders)
React Native	0.86.3	Cross-platform mobile runtime
React	19.2.3	UI library
@react-navigation/native + native-stack	7.5.0 / 7.20.0	Stack navigation between the 3 screens (<code>App.js</code>)
axios	1.20.0	HTTP client (<code>src/api.js</code>)
@react-native-async-storage/async-storage	2.2.0	Persists the JWT token on device
lucide-react-native + react-native-svg	1.49.0 / 15.15.4	Icons
expo-font / expo-splash-screen / expo-status-bar	~57.x	Geist font loading, splash, status bar
react-native-safe-area-context / react-native-screens	~5.7.0 / ~4.26.0	Safe areas and native screen containers
@expo/ngrok	4.1.3	<code>expo start --tunnel</code> for testing on a phone off-LAN

Push notification library: **Not found in the project** (notifications are in-app only, fetched from the API).

Backend (backend/package.json)

Technology	Version	Why it is used
Node.js (ES modules)	"type": "module"	Runtime
Fastify	^5.0.0	Fast, low-overhead HTTP framework
@fastify/jwt	^10.2.2	JWT signing/verification
@fastify/cors	^10.0.1	CORS handling
@fastify/multipart	^9.0.1	CSV file uploads
Prisma + @prisma/client	^5.22.0	ORM / type-safe DB access
@prisma/adapters-neon + @neondatabase/serverless	^5.22.0 / ^0.10.4	Serverless Postgres driver adapter
ws	^8.22.0	WebSocket transport required by Neon driver (src/lib/prisma.js:10)
bcrypt	^6.0.0	Password hashing
csv-parser	^3.0.0	Streaming CSV parsing for uploads
dotenv	^18.0.4	Loads backend/.env
jsonwebtoken	^9.0.3	Listed as dependency — not imported anywhere in backend/src (dead dependency)
pg	^8.23.0	Listed as dependency — not directly imported in backend/src
Vitest + Supertest	^5.0.3 / ^7.3.0 (dev)	API tests (backend/test/auth.test.js)

Database

Technology	Where defined	Why
PostgreSQL (Neon serverless)	backend/prisma/schema.prisma (provider = "postgresql"), backend/src/lib/prisma.js	Main relational database
Prisma Migrate	—	Migrations folder: Not found in the project

DevOps / Deployment

Tool	File	Purpose
Docker + docker-compose	docker-compose.yml , three Dockerfiles	Containerized deployment
nginx (alpine)	frontend/nginx.conf , student-frontend/nginx.conf	Serves built SPA with history fallback
node:20-alpine	all Dockerfiles	Build/run base image
CI/CD pipelines	—	Not found in the project (no .github/, no pipeline config)
localtunnel / ngrok (dev only)	student-mobile/backend-tunnel.log , metro-tunnel.log	Expose backend/Metro to a physical phone during development

Third-party services

Service	Where used	Purpose
Neon (serverless Postgres)	<code>backend/src/lib/prisma.js</code> , <code>DATABASE_URL</code> in <code>backend/.env</code>	Hosted database
AI/LLM, payment, email, SMS, cloud storage	—	Not found in the project

3. WEB FRONTEND

3.1 Admin / Examiner portal (`frontend/`)

- **Framework:** React 19 + Vite (`frontend/vite.config.js`). Plain JSX, no TypeScript (`jsconfig.json`).
- **Routing** (`frontend/src/App.jsx`):

Path	Component
<code>/</code>	Redirects to <code>/login</code>
<code>/login</code>	<code>src/pages/Login.jsx</code>
<code>/admin/*</code>	<code>src/pages/admin/AdminDashboard.jsx</code>
<code>/examiner/*</code>	<code>src/pages/examiner/ExaminerDashboard.jsx</code>

- **Route protection:** No guard component. Inline checks: `AdminLayout.jsx:10-15` redirects to `/` when `localStorage.adminToken` is missing; `ExaminerDashboard.jsx:31-38` redirects to `/login` when `localStorage.examinerToken` is missing, and clears the token on any 401/403 (lines 54-57).
- **State management:** Plain `useState/useEffect` only. Redux/Zustand/Context: **Not found in the project**. Admin state (active batch, tab, semester, refresh trigger) lives in `AdminDashboard.jsx` and is passed as props.
- **UI library:** shadcn-style primitives over `@base-ui/react` in `src/components/ui/` (alert-dialog, avatar, badge, button, card, dialog, dropdown-menu, input, label, select, table, tabs), Tailwind v4 theme in `src/index.css`.
- **Backend calls:** `axios`, with a global request interceptor in `src/main.jsx:7-13` that attaches `Authorization: Bearer <localStorage.adminToken>`.
- **Base URL:** `http://localhost:3000` **hardcoded in 31 places** across `src/`. Environment variable / Vite proxy: **Not found in the project**.
- **Auth flow:** `Login.jsx` has an ADMIN/EXAMINER toggle → `POST /api/auth/admin/login` stores token in `localStorage.adminToken` → `/admin`; `POST /api/auth/examiner/login` stores `localStorage.examinerToken` → `/examiner`; examiner self-sign-up calls `POST /api/auth/examiner/register`. Logout removes the token (`AdminLayout.jsx:147`, `ExaminerDashboard.jsx:61-64`).

Pages & components:

File	What it does	Endpoints called
<code>src/pages/Login.jsx</code>	Shared login with role toggle + examiner registration	<code>POST /api/auth/admin/login</code> , <code>POST /api/auth/examiner/login</code> , <code>POST /api/auth/examiner/register</code>

File	What it does	Endpoints called
src/layouts/AdminLayout.jsx	Admin sidebar shell (batch tabs: GLOBAL_ALLOCATION, GENERAL, COMPUTER, ELECTRICAL, MECHANICAL, CIVIL), mobile drawer, logout	none
src/pages/admin/AdminDashboard.jsx	Orchestrator: holds selected batch/semester/tab state, renders the views below	GET /api/admin/batches, GET /api/admin/batches/:batchId/departments/:dept/stats
src/components/GlobalDashboard.jsx	Batch cards + create/edit/delete batch; auto-downloads generated student passwords as CSV	POST/PUT/DELETE /api/admin/batches(/:id)
src/components/DepartmentWorkspace.jsx	Per-department CSV upload card to allocate students by registration number	GET ../departments/:dept/stats, POST ../departments/:dept/upload
src/components/DepartmentAllocation.jsx	Older duplicate of the allocation UI — dead code, never imported	(same upload endpoint)
src/components/ModuleManagement.jsx	Module CRUD table per semester/department, GPA/NGPA and result-status badges, semester-vs-code validation	GET/POST /api/admin/modules..., PUT/DELETE /api/admin/modules/:code
src/components/ResultUploadModal.jsx	Upload results CSV, preview with repeat/previous grade, publish/unpublish (fake progress bar, lines 52-60)	GET/POST /api/admin/modules/:code/results..., ../publish, ../unpublish
src/components/MasterResultSheet.jsx	Pivoted batch result sheet (grades + SGPA/CGPA per student)	GET /api/admin/batches/:batchId/semesters/:sem/departments/:dept/master-sheet
src/components/ExaminerManagement.jsx	Create examiners, CRUD exam schedules ("Assign & Notify")	GET/POST /api/admin/examiners, GET/POST/PUT/DELETE /api/admin/exam-schedules(/:id)
src/pages/examiner/ExaminerDashboard.jsx	Examiner portal: schedule tab, profile tab, notification bell	GET /api/examiner/schedules, GET/POST /api/examiner/notifications(/read), GET/PUT /api/examiner/profile

3.2 Student portal (student-frontend/)

- Same stack as above (student-frontend/package.json).
- **Routing** (src/App.jsx:11-16): / → Login.jsx, /change-password → ChangePassword.jsx, /dashboard → StudentDashboard.jsx, * → redirect /.
- **Protection:** inline in StudentDashboard.jsx:23-27 (checks localStorage.studentToken, redirects if missing; clears token on 401/403, lines 43-46). /change-password has **no client-side guard**.
- **Backend calls:** axios; http://localhost:3000 hardcoded (Login.jsx:20, StudentDashboard.jsx:31,32,57,88, ChangePassword.jsx:30). Env vars: **Not found in the project**.
- **Login flow:** POST /api/auth/student/login with {regNo, password} → token in localStorage.studentToken → if isFirstLogin go to /change-password, else /dashboard (Login.jsx:20-25). ChangePassword.jsx calls POST /api/auth/student/change-password and replaces the stored token with the new one (line 36).
- **Dashboard:** tabs OVERVIEW (CGPA card, credits out of a hardcoded 150, 5 most recent released results), TIMETABLE, and one tab per semester (SGPA + results table); notification

bell marks all read when opened. Calls `GET /api/student/dashboard`, `GET /api/student/exam-schedules`, `GET /api/student/notifications`, `POST /api/student/notifications/read`.

4. MOBILE APP (`student-mobile/`)

- **Framework:** Expo SDK ~57 managed workflow (React Native 0.86.3, React 19.2.3). No `android/` or `ios/` folders — native projects are generated (CNG), per `student-mobile/AGENTS.md`. Note: `AGENTS.md` says to use Expo Router with `src/app/`, but the app actually uses **@react-navigation/native-stack** with `src/screens/` — documentation drift.
- **Navigation** (`App.js:41-47`): `NavigationContainer` + native stack with three screens, all `headerShown:false`: `Login` (initial) → `ChangePassword` (if `isFirstLogin`, via `navigation.replace`, `LoginScreen.js:33-37`) → `Dashboard`. `Logout` replaces back to `Login` (`DashboardScreen.js:68`); `Profile` navigates to `ChangePassword`.
- **State management:** local `useState` + `useCallback`; data refetched on screen focus via `useFocusEffect` (`DashboardScreen.js:87-92`). No `Redux/Context`: **Not found in the project**.
- **Local storage:** `AsyncStorage` key `studentToken` (set in `LoginScreen.js:30`, `ChangePasswordScreen.js:37`; removed in `DashboardScreen.js:68,95`).
- **Backend calls:** single `axios` instance in `src/api.js` (timeout 12000 ms) with a request interceptor that injects `Authorization: Bearer <token>` from `AsyncStorage`.
- **Base URL** (`src/config.js`), resolution order: 1. `EXPO_PUBLIC_API_URL` env var (from `student-mobile/.env`); 2. else the Metro dev-server hostname → `http://<hostname>:3000` (auto LAN-IP detection for physical phones; tunnel hostnames like `ngrok/loca` are ignored); 3. else `http://localhost:3000`.
- **Screens:**

Screen	File	Endpoints
Login	<code>src/screens/LoginScreen.js</code>	<code>POST /api/auth/student/login</code>
Change Password	<code>src/screens/ChangePasswordScreen.js</code>	<code>POST /api/auth/student/change-password</code>
Dashboard	<code>src/screens/DashboardScreen.js</code> (1096 lines)	<code>GET /api/student/dashboard</code> , <code>GET /api/student/exam-schedules</code> , <code>GET /api/student/notifications</code> , <code>POST /api/student/notifications/read</code>

Dashboard mirrors the web portal: OVERVIEW / TIMETABLE / semester tabs, CGPA gradient card, recent results with New/Repeat badges, notification modal, `SidebarDrawer` (custom component) with profile + logout. - **Platform-specific code:** `app.json` — portrait orientation, splash background `#611010`, Android adaptive icon, iOS `supportsTablet: true`, plugins `expo-font` (Geist) and `expo-splash-screen`. No explicit Android/iOS permissions declared. Android rendering fix documented in `docs/Progress.md` Phase 5 (`elevation` + `overflow:hidden` must be on separate Views — `DashboardScreen.js` clip layers `cgpaClip/examClip/notifClip`). - **Push notifications: Not found in the project** (listed as optional backlog in `docs/Progress.md` §4). - **Build/run:** scripts are only `expo start` variants (`package.json:22-27`). EAS config (`eas.json`): **Not found in the project**. Dev logs show tunnel usage: `backend-tunnel.log` (localtunnel URL for the backend) and `metro-tunnel.log` (`expo start --tunnel`).

5. BACKEND

5.1 Framework & architecture

- **Fastify 5**, ES modules. Entry point: `backend/src/server.js`.
- Architecture: **route-based, single-layer** — each route handler contains its own business logic and calls Prisma directly. There is no separate controller/service/repository layer: **Not found in the project**.
- Plugins registered (`server.js:31-38`): `@fastify/cors` (`origin: '*'`), `@fastify/multipart`, `@fastify/jwt` with **hardcoded secret** `'supersecret_ruhuna_key'` (`server.js:37`, with a comment "In production, use env var").
- Auth hook: `fastify.authenticate` (`server.js:41-47`) runs `request.jwtVerify()` and is applied as an `onRequest` hook to the `/api/admin`, `/api/student`, `/api/examiner` prefixes (`server.js:50-62`). `/api/auth` is open (except `change-password`, which verifies the JWT itself in `authRoutes.js:150`).
- On startup the server seeds 5 departments (GENERAL, COMPUTER, ELECTRICAL, MECHANICAL, CIVIL) with upserts (`server.js:14-29`) and listens on `0.0.0.0:3000` (`server.js:67`).
- Helper `generateNewStudents()` (`adminRoutes.js:5-42`): expands a registration-number range (e.g. `EG/2024/001` → `EG/2024/150`), generates random 8-char passwords with `Math.random().toString(36).slice(-8)`, hashes with `bcrypt` cost **6** ("Reduced for faster bulk generation", line 28), and returns plaintext passwords once.

5.2 Validation & error handling

- Manual validation inside handlers (missing fields → 400, Prisma `P2002` unique errors → 400 with friendly message, e.g. `adminRoutes.js:65-67`). No schema validation library (`zod/typebox/ajv`): **Not found in the project**.
- Every handler has `try/catch` → `fastify.log.error` + 500 JSON error.
- Role checks: student routes check `role !== 'STUDENT'` → 403 (e.g. `studentRoutes.js:8`), examiner routes check `role !== 'EXAMINER'` → 403. **Admin routes never check** `role === 'ADMIN'` — any valid JWT (even a student's) passes (see §11).

5.3 Complete API endpoint table

Auth endpoints (prefix `/api/auth`, file `backend/src/routes/authRoutes.js`):

Method	URL	Purpose	Request body	Response	Auth needed
POST	<code>/api/auth/admin/login</code>	Admin login	<code>{email, password}</code>	<code>{success, token, email}</code>	No
POST	<code>/api/auth/student/login</code>	Student login	<code>{regNo, password}</code>	<code>{success, token, regNo, isFirstLogin}</code>	No
POST	<code>/api/auth/examiner/login</code>	Examiner login (403 if not registered)	<code>{email, password}</code>	<code>{success, token, email}</code>	No
POST	<code>/api/auth/examiner/register</code>	Examiner sets own password (account must pre-exist)	<code>{email, password}</code>	<code>{success, message}</code>	No

Method	URL	Purpose	Request body	Response	Auth needed
POST	/api/auth/student/change-password	Forced first-login password change; issues new JWT	{newPassword} (min 6 chars)	{success, token, regNo, isFirstLogin:false}	Yes (STUDENT JWT)

Admin endpoints (prefix /api/admin, file backend/src/routes/adminRoutes.js, JWT required, **role not checked**):

Method	URL	Purpose	Request	Response	Auth
POST	/api/admin/batches	Create batch + auto-generate students & passwords	{name, startRegNo, endRegNo}	201 batch + newStudentPasswords[] (plaintext, one-time)	JWT
PATCH	/api/admin/batches/:batchId	Update batch, generate any new students	{name, startRegNo, endRegNo}	batch + new StudentPasswords[]	JWT
DELETE	/api/admin/batches/:batchId	Delete batch (students cascade-delete)	—	{success:true}	JWT
GET	/api/admin/batches	List batches (newest first)	—	Batch[]	JWT
POST	/api/admin/batches/:batchId/departments/:department/upload	Allocate students to a department from CSV (regex-extracts EG xxxx_xxxx reg nos; skips students already in another dept)	multipart file	{success, count, message, skipped}	JWT
GET	/api/admin/batches/:batchId/departments/:department/stats	Student count per department	—	{studentCount}	JWT
GET	/api/admin/batches/:batchId/semesters/:semester/departments/:department/master-sheet	Pivoted master result sheet with SGPA/CGPA/total credits per student (4.0 scale, repeat grades * capped at 2.0, NGPA excluded)	—	{modules, pivotData}	JWT
POST	/api/admin/modules	Create module; credits derived from 2nd digit of code; dept-prefix rules (EC/ME/EE/CE)	{code, name, type, semester, department, isGpa}	201 Module	JWT
GET	/api/admin/modules/:semester/departments/:department?batchId=	List modules with result status (PUBLISHED/DRAFT/NO_RESULTS) and count	—	mapped module list	JWT
PATCH	/api/admin/modules/:code	Update module (note: isGpa defaults to true when omitted, line 286)	{name, type, isGpa}	Module	JWT

Method	URL	Purpose	Request	Response	Auth
DELETE	/api/admin/modules/:code	Delete module (results cascade)	—	{success:true}	JWT
POST	/api/admin/modules/:code/results/upload?batchId=&isRepeatUpload=	Upload results CSV (columns parsed: reg no pattern + grade/result/marks/mark + optional status/type/gpa/... for GPA/NGPA). Upserts as drafts, marks repeats (cross-batch or flag), stores previousGrade (defaults 'F'), chunks writes in transactions of 100	multipart file	{success, count}	JWT
GET	/api/admin/modules/:code/results?department=	List results for a module (includes student)	—	Result[]	JWT
POST	/api/admin/modules/:code/results/publish	Publish all draft results of a module + create per-student notifications	—	{success:true}	JWT
POST	/api/admin/modules/:code/results/unpublish	Unpublish all results of a module + delete related notifications (matched by message text containing module <code>)	—	{success:true}	JWT
POST	/api/admin/examiners	Create examiner stub (isRegistered:false)	{email, name, departmentId}	201 {email, name}	JWT
GET	/api/admin/examiners	List examiners (no passwords)	—	Examiner[]	JWT
POST	/api/admin/exam-schedules	Schedule exam + notify examiner ("New Exam Duty Assigned")	{moduleCode, examinerEmail, date, venue}	201 ExamSchedule	JWT
PATCH	/api/admin/exam-schedules/:id	Update schedule (no notification sent)	same	ExamSchedule	JWT
DELETE	/api/admin/exam-schedules/:id	Delete schedule	—	{success:true}	JWT
GET	/api/admin/exam-schedules	List all schedules with module + examiner	—	ExamSchedule[]	JWT

Student endpoints (prefix /api/student , file backend/src/routes/studentRoutes.js , JWT + STUDENT role):

Method	URL	Purpose	Response	Auth
GET	/api/student/dashboard	Results grouped by semester (own dept + GENERAL modules), SGPA per semester, CGPA; only published results count toward GPA	{student, resultsBySemester, sgpaBySemester, cgpa}	STUDENT JWT
GET	/api/student/notifications	List own notifications (newest first)	Notification[]	STUDENT JWT

Method	URL	Purpose	Response	Auth
POST	/api/student/notifications/read	Mark all own notifications read	{success:true}	STUDENT JWT
GET	/api/student/exam-schedules	Exam schedules for own dept + GENERAL modules, with module info	ExamSchedule[]	STUDENT JWT

Examiner endpoints (prefix /api/examiner, file backend/src/routes/examinerRoutes.js, JWT + EXAMINER role):

Method	URL	Purpose	Response	Auth
GET	/api/examiner/schedules	Own assigned exam schedules with module	ExamSchedule[]	EXAMINER JWT
GET	/api/examiner/notifications	Own notifications	ExaminerNotification[]	EXAMINER JWT
POST	/api/examiner/notifications/read	Mark all read	{success:true}	EXAMINER JWT
GET	/api/examiner/profile	Own profile (password stripped, line 76)	Examiner	EXAMINER JWT
PUT	/api/examiner/profile	Update name and/or password (bcrypt via dynamic import, lines 96-101)	{success:true}	EXAMINER JWT

5.4 Authentication & authorization

- **JWT stateless auth.** Tokens signed by @fastify/jwt with roles embedded: ADMIN, STUDENT (includes regNo, batchId, department, isFirstLogin), EXAMINER (authRoutes.js:24-28, 61-68, 101-106).
- Passwords hashed with bcrypt (cost 10 normally; cost 6 for bulk student generation, adminRoutes.js:28).
- Token expiry config: **Not found in the project** (default @fastify/jwt behavior — no expiration set).
- Sessions/refresh tokens: **Not found in the project.**
- Authorization is by role claim inside each handler — but missing on all admin routes (see §11).
- Admin user creation: there is **no admin registration or seed script** — **Not found in the project** (an Admin row must be inserted into the database manually).

5.5 Background jobs / schedulers / workflows

- **Not found in the project.** No cron, queue, worker, or n8n integration exists. All notifications are created synchronously inside request handlers (adminRoutes.js:488-501, 726-732).

5.6 Tests

- backend/test/auth.test.js — Vitest + Supertest, 6 tests covering 400/401 paths of the auth endpoints. Run with npm test (backend/package.json). No tests for admin/student/examiner routes.

6. DATABASE

6.1 Type & connection

- **PostgreSQL** hosted on **Neon serverless** (backend/prisma/schema.prisma:6-9).
- Connection: DATABASE_URL from backend/.env → Neon Pool (WebSocket transport via ws) → PrismaNeon adapter → PrismaClient (backend/src/lib/prisma.js). Preview feature driverAdapters enabled (schema.prisma:3).
- Note: backend/prisma/dev.db (40 KB SQLite file) still exists — a leftover from an earlier SQLite setup. The current schema targets Postgres, so this file is unused by the running app.

6.2 Tables (Prisma models, all in backend/prisma/schema.prisma)

Model	Primary key	Columns (type)	Foreign keys / relations
Department	id (String, e.g. GENERAL, COMPUTER)	name	— (referenced by Examiner, Module, Result, Student)
Batch	id (uuid)	name (unique), startRegNo, endRegNo, createdAt, updatedAt	1→many Students
Student	regNo (String, e.g. EG/2024/001)	batchId, departmentId (default GENERAL), password (nullable), isFirstLogin (default true), createdAt, updatedAt	→ Batch (cascade delete), → Department; indexes on batchId, departmentId
Module	code (String, e.g. EC 3201)	name, credits (default 3), type, isGpa (default true), semester (Int), departmentId (default GENERAL), timestamps	→ Department; indexes on departmentId, semester
Admin	id (uuid)	email (unique), password, timestamps	—
Result	composite (studentRegNo, moduleCode)	departmentId (default GENERAL), grade, previousGrade (nullable), isGpa (nullable = use Module default), isRepeat (default false), isPublished (default false), timestamps	→ Module (cascade), → Student (cascade), → Department; indexes on moduleCode, isPublished, departmentId
Notification	id (uuid)	studentRegNo, title, message, isRead (default false), createdAt	→ Student (cascade); index (studentRegNo, isRead)
Examiner	email	name, password (nullable), departmentId (default GENERAL), isRegistered (default false), timestamps	→ Department
ExamSchedule	id (uuid)	moduleCode, examinerEmail, date (DateTime), venue, timestamps	→ Examiner (cascade), → Module (cascade); indexes on examinerEmail, moduleCode, date
ExaminerNotification	id (uuid)	examinerEmail, title, message, isRead, createdAt	→ Examiner (cascade); index (examinerEmail, isRead)

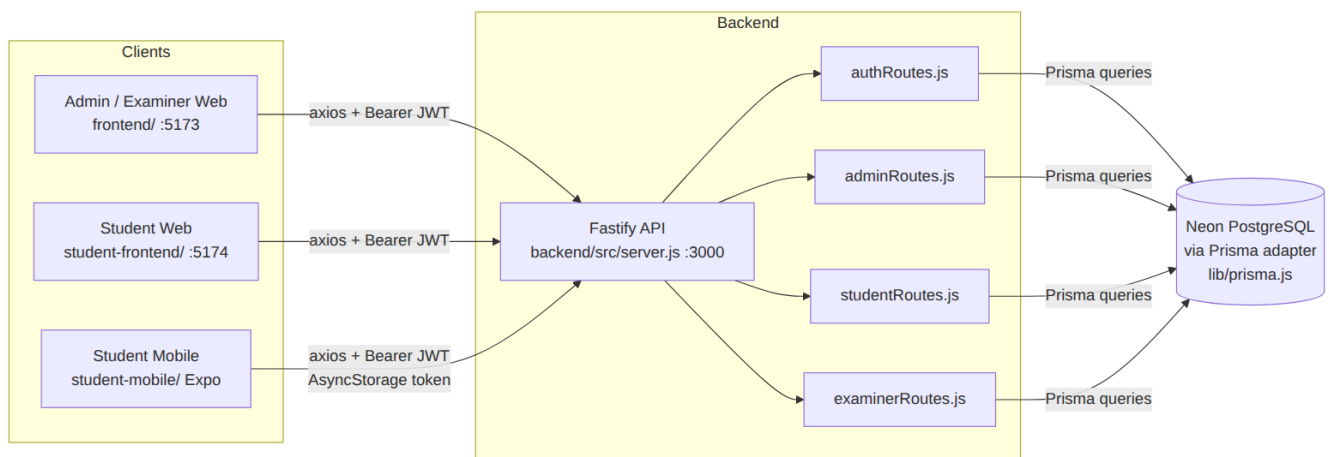
Relationship summary: Batch → Students; Department → Students/Modules/Examiners/Results; Student+Module → Result; Student → Notifications; Examiner → ExamSchedules + ExaminerNotifications; Module → ExamSchedules.

6.3 Migrations & seeds

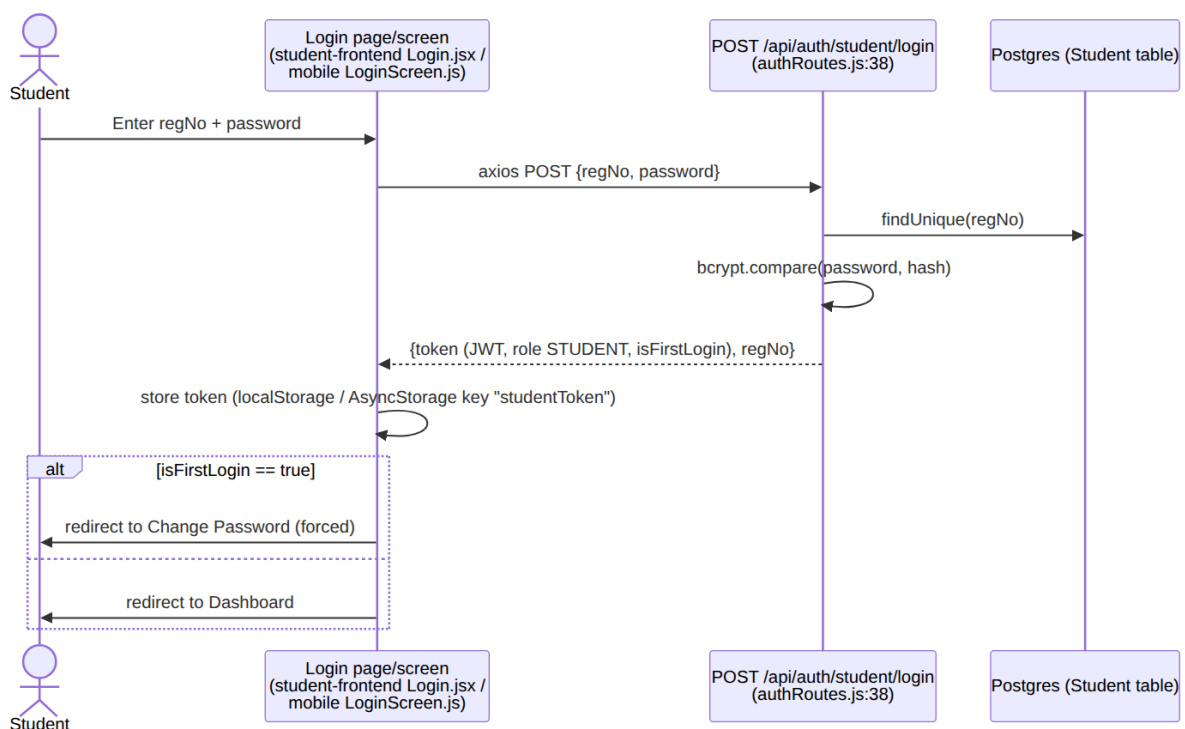
- **Prisma migrations folder: Not found in the project** (schema is presumably applied with `prisma db push`).
- Seed data: the 5 departments are upserted on every server start (`backend/src/server.js:14-29`). No other seed script — **Not found in the project**.
- `backend/regenerate_passwords.cjs` : one-off script that regenerates all passwords of the "24th Batch" and writes `../<Batch>_Passwords.csv` (this produced the CSV files in the project root).

7. DATA FLOW

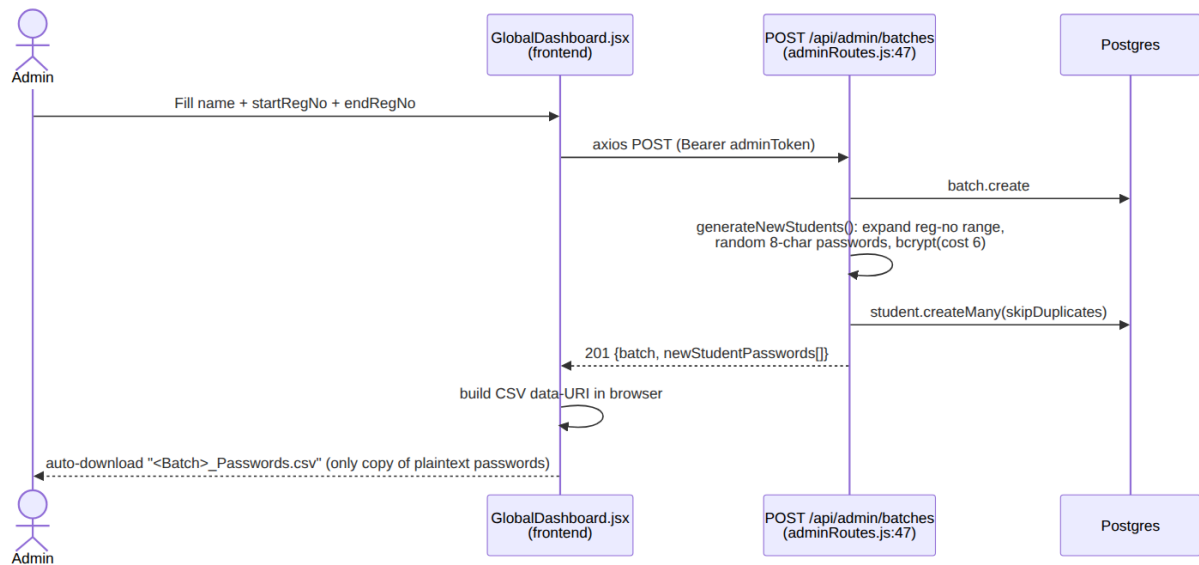
7.1 Overall architecture



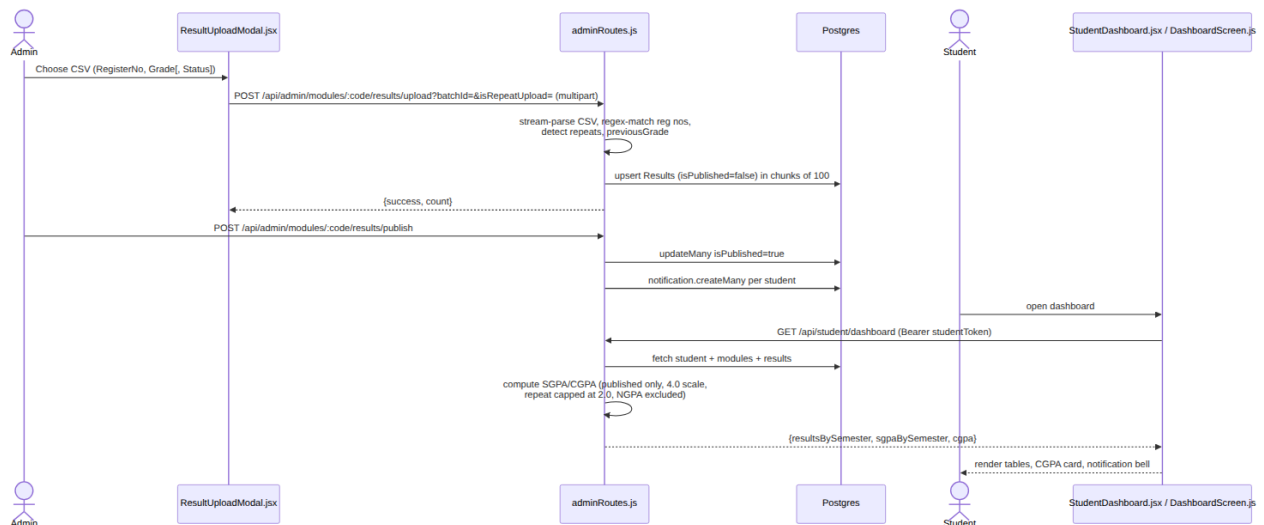
7.2 Flow 1 — Student login (Web and Mobile, same API)



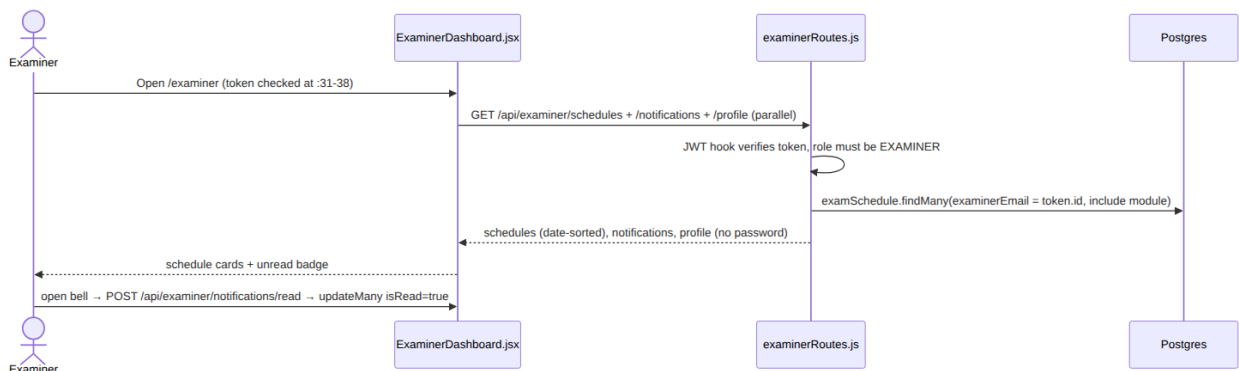
7.3 Flow 2 — Admin creates a batch (create/save flow)



7.4 Flow 3 — Result upload & publish (admin write → student read)



7.5 Flow 4 — Read/list flow: examiner views schedule



8. EXTERNAL INTEGRATIONS

Integration	Where	Keys/credentials
Neon serverless Postgres	<code>backend/src/lib/prisma.js</code> (Pool + PrismaNeon adapter + <code>ws</code>)	<code>DATABASE_URL</code> in <code>backend/.env</code> (connection string; value not printed here)
localtunnel (dev only)	exposes backend for physical-phone testing (<code>student-mobile/backend-tunnel.log</code> , instructions in <code>docs/Progress.md §3</code>)	none
ngrok via @expo/ngrok (dev only)	<code>expo start --tunnel</code> (<code>student-mobile/metro-tunnel.log</code>)	none stored
AI/LLM services	Not found in the project	—
Payment gateways	Not found in the project	—
Email / SMS services	Not found in the project — all "notifications" are in-app DB rows only	—
Cloud file storage	Not found in the project — CSV uploads are parsed in memory and discarded	—
Hardcoded JWT secret	<code>backend/src/server.js:37</code> (<code>'supersecret_ruhuna_key'</code>) — should be an env var	value visible in source (a code-quality/security problem, see §11)

9. CONFIGURATION AND SETUP

9.1 Environment variables (names only)

Variable	Where used	Purpose
<code>DATABASE_URL</code>	<code>backend/.env</code> → <code>prisma/schema.prisma</code> , <code>src/lib/prisma.js</code>	Neon Postgres connection string
<code>EXPO_PUBLIC_API_URL</code>	<code>student-mobile/.env</code> (optional) → <code>src/config.js</code>	Override backend base URL on the phone
<code>NODE_ENV</code>	<code>docker-compose.yml</code> (set to <code>production</code> for backend)	Runtime mode

Web frontends use **no env variables** (base URL hardcoded).

9.2 Run locally

```
# Backend (port 3000)
cd backend
npm install
npx prisma generate      # + schema push to your Neon DB (no migrations folder exists)
npm run dev              # node --watch src/server.js (or: npm start)
npm test                 # vitest run (auth tests)

# Admin/Examiner web (port 5173 default)
cd frontend
npm install
npm run dev

# Student web (port 5174, from package.json script)
cd student-frontend
npm install
npm run dev

# Student mobile
cd student-mobile
npm install
npx expo start           # scan QR with Expo Go; phone must reach backend
# Emulator: adb reverse tcp:8081 tcp:8081 && adb reverse tcp:3000 tcp:3000
# Off-LAN phone: tunnel backend (npx localtunnel --port 3000),
# then set EXPO_PUBLIC_API_URL in student-mobile/.env
```

(Mobile instructions verified against `docs/Progress.md` §3.)

9.3 Build

- Web apps: `npm run build` → `dist/` (Vite). Lint: `npm run lint` (oxlint).
- Mobile: `npx expo export --platform android` verified per `docs/Progress.md` Phase 4. Release APK via `npx expo run:android` or EAS Build — EAS config **Not found in the project**.

9.4 Docker / deployment

- `docker-compose.yml` (project root): three services —
- `backend`: builds `backend/Dockerfile` (`node:20-alpine` + `openssl`, `prisma generate`, `npm start`), port **3000:3000**, reads `backend/.env`, `NODE_ENV=production`.
- `frontend`: builds to static, served by `nginx:alpine`, port **5173:80**, `depends_on`: `backend`.
- `student-frontend`: same pattern, port **5174:80**.
- **Known deployment gap**: both `nginx.conf` files only serve the SPA (`try_files ... /index.html`) with **no /api reverse proxy**, and the frontends hardcode `http://localhost:3000`. In Docker, the browser would call the *user's* localhost — the deployed web apps cannot reach the backend without code/config changes (see §11).
- CI/CD: **Not found in the project**.

10. PROJECT HISTORY AND STATUS

10.1 Git history

Not found in the project — the folder is not a git repository (`git log` fails with "not a git repository"). Milestones below come from `docs/Progress.md` (last updated 2026-10-02).

10.2 Milestones (from docs/Progress.md)

Phase	What was built	Status
1	Backend API: Fastify 5, JWT, bcrypt, all admin/student/auth endpoints, Prisma schema on Neon	✓ Complete
2	Admin/Examiner web portal (React 19, Vite, Tailwind v4, shadcn/ui)	✓ Complete
3	Student web portal with brand theme (maroon #611010 / gold #f5bd1a), login, forced password change, dashboard	✓ Complete
4	Student mobile app (Expo SDK 57): pixel-parity port of the student web portal, LAN auto-detect, production bundle verified	✓ Complete
5	Android rendering fixes (elevation/overflow bug), safe-area handling, responsive sizing, emulator QA (Pixel 7, Android 15), feature-parity audit vs web	✓ Complete (current milestone)

10.3 Who worked on what

No author metadata in code (package.json "author" fields are empty; no git history). From workspace records of prior conversations: the project belongs to GitHub user **Chathurdha2024**, and the current user (Dewmini Denuwanthi / Denuwanthi2k02) contributes to the **student web part**. No other contributor info — otherwise **Not found in the project**.

10.4 Finished vs. half-done vs. missing

- **Finished:** all endpoints listed in §5, both web portals, mobile app parity, Docker files, department seeding, auth tests.
- **Half-done / acknowledged backlog** (docs/Progress.md §4):
 - On-device QA of mobile golden paths (emulator done, physical phone pending).
 - Branded app icon & splash (Expo defaults now).
 - Push notifications (Expo Notifications) — not started.
 - Release APK via EAS Build — not configured.
 - Backend hardening: rate limiting, helmet, JWT secret to env — not done (secret still hardcoded, server.js:37).
- **TODO/FIXME comments in source:** none found in any of the four apps.
- **Missing entirely:** CI/CD, Prisma migrations, admin user seeding, chart library (despite CSS chart variables), nginx API proxy, E2E tests (only recommended in docs/qa_documentation.md).

11. PROBLEMS AND IMPROVEMENTS

11.1 Problems found in the code

Security (most important first):

1. **Admin routes have no role check.** The JWT hook (server.js:52) only verifies the token is valid; adminRoutes.js never checks role === 'ADMIN' . A logged-in **student or examiner can call every admin endpoint** (create batches, publish results, read all students' data). Contrast: student/examiner routes do check roles (studentRoutes.js:8 , examinerRoutes.js:6).
2. **JWT secret hardcoded** in backend/src/server.js:37 ('supersecret_ruhuna_key') — anyone with repo access can forge admin tokens.

3. **Plaintext password CSVs in the project root** (`25th_Batch_Passwords (1).csv` , `26th_Batch_Passwords.csv` , ~10 KB of real registration numbers + passwords). These should never be stored or shared in the repo folder.
4. **CORS origin: '*'** (`server.js:32-34`) — any website can call the API from a browser.
5. **No rate limiting / helmet** — login endpoints are brute-forceable (already acknowledged in `docs/Progress.md §4` and `docs/qa_documentation.md`).
6. **Weak password generation**: `Math.random().toString(36).slice(-8)` (`adminRoutes.js:27`) is not cryptographically secure; bcrypt cost 6 for bulk students is below the recommended 10+.
7. **JWTs never expire** (no `expiresIn` anywhere) and logout is client-side only.
8. **Hardcoded http://localhost:3000** in both web apps (31 occurrences in `frontend/src` , 5 in `student-frontend/src`) with no nginx `/api` proxy — the Docker deployment is broken as configured, and any future HTTPS site would face mixed-content/CORS errors.

Code quality:

9. Dead code: `frontend/src/components/DepartmentAllocation.jsx` (never imported); unused `modules state` (`ExaminerManagement.jsx:15`); unused `semester prop` (`DepartmentWorkspace.jsx:8`); duplicated `setBatches(res.data)` (`AdminDashboard.jsx:47-48`); unused `ui/` primitives in both web apps; `jsonwebtoken` and `pg` dependencies unused in backend source.
10. Fragile logic: `unpublish` deletes notifications by **message text match** `contains: "module <code>e>"` (`adminRoutes.js:520-522`) — a module code appearing in other messages would delete unrelated notifications. `PUT /api/admin/modules/:code` resets `isGpa` to `true` when the field is omitted (`adminRoutes.js:286`). Misleading "type any dummy credentials" copy on both login pages (`frontend/src/pages/Login.jsx:159-162` , `student-frontend/src/pages/Login.jsx:100`). Fake upload progress bar (`ResultUploadModal.jsx:52-60`). Brittle "run directly" check for Windows paths (`server.js:79`). Global axios interceptor only injects `adminToken` , so examiner pages must hand-roll headers (`frontend/src/main.jsx:7-13`). `expo-constants` used but not declared in `student-mobile/package.json` . Student web `/change-password` route renders without a token check. Duplicated GPA logic in three places (`studentRoutes.js:13-28` , `adminRoutes.js:554-570` , and client-side credit math in both student apps); hardcoded 150-credit denominator (`StudentDashboard.jsx:371` , `DashboardScreen.js:174`). Leftover `backend/prisma/dev.db` SQLite file while schema targets Postgres. `student-mobile/AGENTS.md` documents Expo Router but the app uses react-navigation.
11. Minimal tests: only 6 auth-endpoint tests (`backend/test/auth.test.js`); no tests for the 21 admin endpoints, GPA engine, CSV parsing, or any UI.

11.2 Top 10 improvements

1. **Add a role check to all /api/admin routes** (e.g. an `onRequest` hook asserting `request.user.role === 'ADMIN'`) — highest-severity fix.
2. **Move the JWT secret to an env var** (`JWT_SECRET`), set token expiry (e.g. 8h), and rotate the current secret.
3. **Delete the plaintext password CSVs** from the project folder; regenerate and distribute credentials securely, then force `isFirstLogin` resets.
4. **Introduce a single API base URL config** in both web apps (`import.meta.env.VITE_API_URL`) and add an `/api` reverse proxy to both `nginx.conf` files so the Docker stack works end-to-end.
5. **Restrict CORS** to the actual frontend origins instead of `'*'` , and add `@fastify/helmet` + `@fastify/rate-limit` (especially on `/api/auth/*`).
6. **Adopt Prisma migrations** (`prisma migrate dev`) and add an admin seed script, so environments are reproducible; remove the stale `dev.db` .

7. **Extract the GPA/grading-scale logic into one shared module** (`backend/src/lib/grading.js`) used by both `studentRoutes.js` and `adminRoutes.js` , with unit tests (this is exactly what `docs/qa_documentation.md` §3 recommends).
 8. **Replace notification deletion by message-text match** with a proper link (e.g. add `moduleCode` to `Notification`) so unpublish is precise.
 9. **Use `crypto.randomBytes` for generated passwords**, raise bcrypt cost (≥ 10) or hash lazily, and add tests for the admin routes and CSV upload parsing.
 10. **Clean up dead code and drift**: delete `DepartmentAllocation.jsx` , unused state/props/deps, fix the misleading login copy, align `student-mobile/AGENTS.md` with the actual navigation approach, and add EAS configuration + CI (lint/test/build) via GitHub Actions once the repo is under git.
-

End of documentation. Generated from direct source inspection on 2026-10-03; file paths are relative to `G:\ResultManagementSystem`.